

Name: __HAN Zhuo__

EID: __zhuohan3__

CS5489 - Tutorial 2

Gender Classification from Face Images

In this tutorial you will train a classifier to predict whether a face image is male or female.

First we need to initialize Python. Run the below cell.

```
In [59]: %matplotlib inline
import IPython.core.display
import matplotlib_inline # setup output image format
matplotlib_inline.backend_inline.set_matplotlib_formats('svg')
import matplotlib.pyplot as plt
import matplotlib
from numpy import *
from sklearn import *
import os
import zipfile
import fnmatch
random.seed(100)
```

Loading Data and Pre-processing

We first need to load the images. Download `photos-bw.zip` and put it in the same directory as this ipynb file. **Do not unzip the file.** Then run the following cell to load the images.

```
In [60]: imgdata = []
genders = []

# load the zip file
filename = 'photos-bw.zip'
zfile = zipfile.ZipFile(filename, 'r')

for name in zfile.namelist():
    # check file name matches
    if fnmatch.fnmatch(name, "photos-bw/*.png"):
        print("loading", name)
        # open file in memory, and parse as an image
        myfile = zfile.open(name)
        img = matplotlib.image.imread(myfile)
        myfile.close()

    # append to data
    imgdata.append(img)
    genders.append( int(name[len("photos-bw/")] == 'm') ) # 0 is fem
```

```
zfile.close()
imgsize = img.shape
print("DONE: loaded {} images".format(len(imgdata)))
```

```
loading photos-bw/f-039-01.png
loading photos-bw/f-040-01.png
loading photos-bw/f-041-01.png
loading photos-bw/f-042-01.png
loading photos-bw/f-043-01.png
loading photos-bw/f1-001-0.png
loading photos-bw/f1-002-0.png
loading photos-bw/f1-003-0.png
loading photos-bw/f1-004-0.png
loading photos-bw/f1-005-0.png
loading photos-bw/f1-006-0.png
loading photos-bw/f1-007-0.png
loading photos-bw/f1-008-0.png
loading photos-bw/f1-009-0.png
loading photos-bw/f1-010-0.png
loading photos-bw/f1-011-0.png
loading photos-bw/f1-012-0.png
loading photos-bw/f1-013-0.png
loading photos-bw/f1-014-0.png
loading photos-bw/f1-015-0.png
loading photos-bw/m-063-01.png
loading photos-bw/m-064-01.png
loading photos-bw/m-065-01.png
loading photos-bw/m-067-01.png
loading photos-bw/m-069-01.png
loading photos-bw/m-070-01.png
loading photos-bw/m-073-01.png
loading photos-bw/m-074-01.png
loading photos-bw/m-075-01.png
loading photos-bw/m-077-01.png
loading photos-bw/m-083-01.png
loading photos-bw/m-085-01.png
loading photos-bw/m-091-01.png
loading photos-bw/m-097-01.png
loading photos-bw/m-100-01.png
loading photos-bw/m1-003-0.png
loading photos-bw/m1-004-0.png
loading photos-bw/m1-005-0.png
loading photos-bw/m1-008-0.png
loading photos-bw/m1-010-0.png
loading photos-bw/m1-011-0.png
loading photos-bw/m1-012-0.png
loading photos-bw/m1-013-0.png
loading photos-bw/m1-014-0.png
loading photos-bw/m1-015-0.png
loading photos-bw/m1-016-0.png
loading photos-bw/m1-017-0.png
loading photos-bw/m1-018-0.png
loading photos-bw/m1-035-0.png
loading photos-bw/m1-041-0.png
DONE: loaded 50 images
```

Each image is a 45x40 array of pixel values. Run the below code to show an example:

```
In [61]: print(img.shape)
plt.imshow(img, cmap='gray', interpolation='nearest')
plt.show()
```

(45, 40)



Run the below code to show all the images!

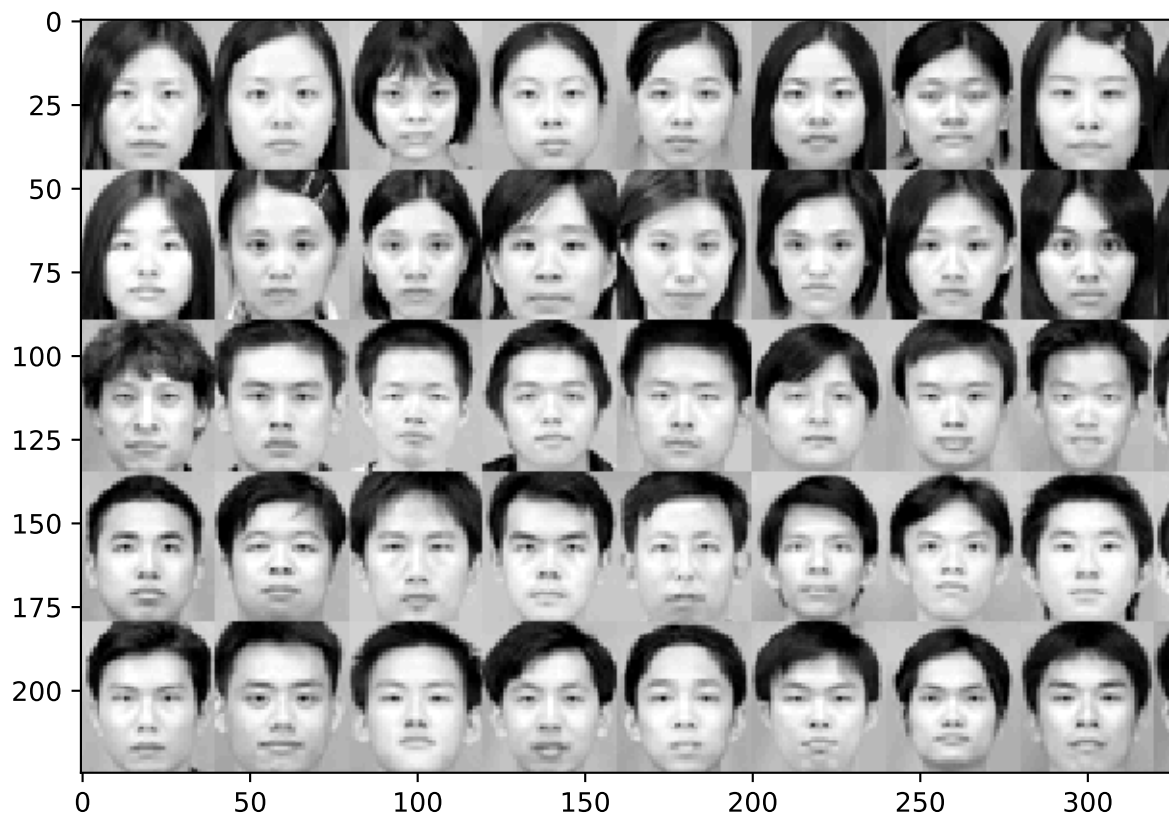
```
In [62]: # function to make an image montage
def image_montage(X, imsize=None, maxw=10):
    """X can be a list of images, or a matrix of vectorized images.
    Specify imsize when X is a matrix."""
    tmp = []
    numimgs = len(X)

    # create a list of images (reshape if necessary)
    for i in range(0, numimgs):
        if imsize != None:
            tmp.append(X[i].reshape(imsize))
        else:
            tmp.append(X[i])

    # add blanks
    if (numimgs > maxw) and (mod(numimgs, maxw) > 0):
        leftover = maxw - mod(numimgs, maxw)
        meaning = 0.5*(X[0].max()+X[0].min())
        for i in range(0, leftover):
            tmp.append(ones(tmp[0].shape)*meaning)

    # make the montage
    tmp2 = []
    for i in range(0, len(tmp), maxw):
        tmp2.append( hstack(tmp[i:i+maxw]) )
    montimg = vstack(tmp2)
    return montimg
```

```
plt.figure(figsize=(9,9))
plt.imshow(image_montage(imgdata), cmap='gray', interpolation='nearest')
plt.show()
```



Each image is a 2d array, but the classifier algorithms work on 1d vectors. Run the following code to convert all the images into 1d vectors by flattening. The result should be a matrix where each row is a flattened image.

```
In [63]: X = empty((50, prod(imgdata[0].shape))) # create empty array
for i,img in enumerate(imgdata):
    X[i,:] = ravel(img) # for each image, turn it into a vector
Y = asarray(genders) # convert list to numpy array
print(X.shape)
print(Y.shape)
```

```
(50, 1800)
(50,)
```

Next we will shift the pixel values so that gray is 0.0, black is -0.5 and white is 0.5.

```
In [64]: print("Before: min={}, max={}".format(X.min(), X.max()))
X -= 0.5
print("After: min={}, max={}".format(X.min(), X.max()))
```

```
Before: min=0.0, max=1.0
After: min=-0.5, max=0.5
```

Finally, split the dataset into a training set and testing set. We select 80% for training and 20% for testing.

```
In [65]: # randomly split data into 80% train and 20% test set
trainX, testX, trainY, testY = \
    model_selection.train_test_split(X, Y,
    train_size=0.80, test_size=0.20, random_state=4487)

print(trainX.shape)
print(testX.shape)
```

```
(40, 1800)
```

```
(10, 1800)
```

```
In [66]: # compute the average image for later
avgX = mean(X,axis=0)
```

Logistic Regression

Train a logistic regression classifier. Use cross-validation to select the best C parameter.

NOTE: If you are running on DeepDive server, the number of effective jobs is 4, i.e.

`n_jobs=-1` will use 4 CPUs.

```
In [67]: ### INSERT YOUR CODE HERE
logreg = linear_model.LogisticRegression(C=100)
logreg.fit(trainX, trainY)

print("w =", logreg.coef_)
print("b =", logreg.intercept_)
```

```
w = [[-0.03124244 -0.02681914 -0.02562224 ...  0.03342295  0.01827337
      0.04823561]]
```

```
b = [-0.12511381]
```

```
In [68]: predY = logreg.predict(testX)
accuracy = metrics.accuracy_score(testY, predY)
print("Accuracy on test set: {:.2f}%".format(accuracy*100))
```

```
Accuracy on test set: 90.00%
```

Use the learned model to predict the genders for the training and testing data. What is the accuracy on the training set? What is the accuracy on the testing set?

```
In [69]: ### INSERT YOUR CODE HERE
train_predY = logreg.predict(trainX)
train_accuracy = metrics.accuracy_score(trainY, train_predY)
print("Accuracy on train set: {:.2f}%".format(train_accuracy*100))

predYtest = logreg.predict(testX)
test_accuracy = metrics.accuracy_score(testY, predYtest)
print("Accuracy on test set: {:.2f}%".format(test_accuracy*100))
```

```
Accuracy on train set: 100.00%
```

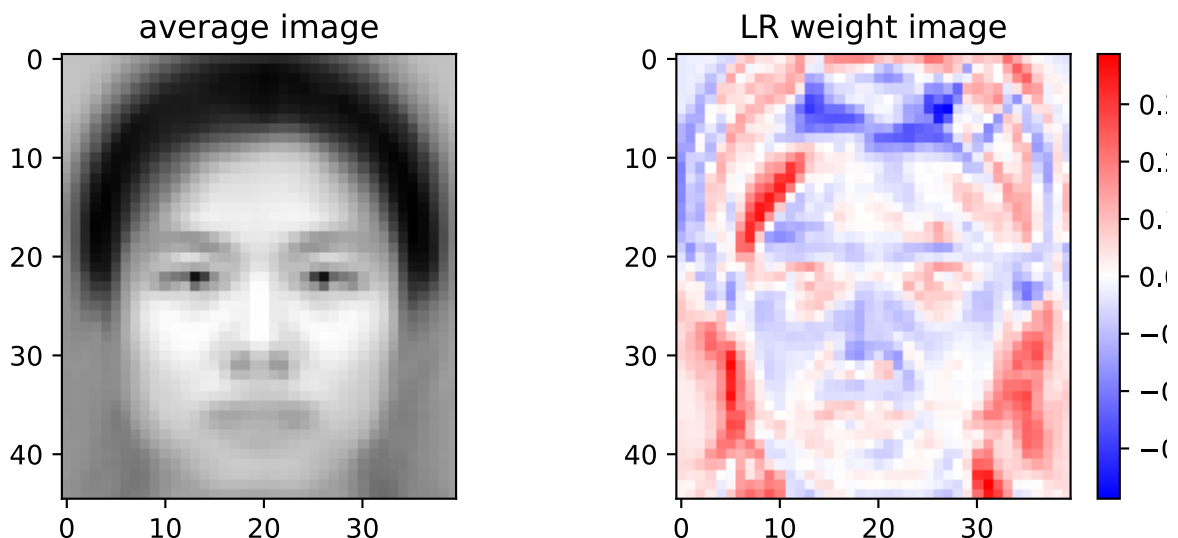
```
Accuracy on test set: 90.00%
```

The accuracy on the training set is 100%, and the accuracy on the testing set is 80%.

Analyzing the classifier

Run the below code to show the hyperplane parameter $\mathbf{w}$ as an image.

```
In [70]: # logreg is the learned logistic regression model
import numpy as np
wimg = logreg.coef_.reshape(imgsize)      # get the w and reshape into an
mycmap = matplotlib.colors.LinearSegmentedColormap.from_list('mycmap', ["
# mm = max(wimg.max(), -wimg.min())
mm = np.abs(wimg).max()
plt.figure(figsize=(8,3))
plt.subplot(1,2,1)
plt.title('average image')
plt.imshow(avgX.reshape(imgsize), cmap='gray')
plt.subplot(1,2,2)
plt.imshow(wimg, interpolation='nearest', cmap=mycmap, vmin=-mm, vmax=mm)
plt.colorbar()
plt.title("LR weight image")
plt.show()
```



Recall that the classifier prediction is based on the sign of the function

$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b = \sum_{i=1}^P w_i x_i + b$. Here each x_i is a pixel in the face image, and w_i is the corresponding weight. Hence, the function is multiplying face image by the weight image, and then summing over all pixels.

In order for $f(\mathbf{x})$ to be positive, then the positive values of the weight image (red regions) should match the positive values in the face image (white pixels), and the negative values of the weight image (blue regions) should be matched with negative values in the face image (black pixels).

Hence, we can have the following interpretation:

Class	red regions (positive weights)	blue regions (negative weights)	white regions (weights near 0)
-------	--------------------------------	---------------------------------	--------------------------------

+1 class (male)	white pixels in face image	black pixels in face image	region not important
-1 class (female)	black pixels in face image	white pixels in face image	region not important

Looking at the weight image, what parts of the face image is the classifier looking at to determine the gender? Does it make sense?

• INSERT YOUR ANSWER HERE

1. The classifier mainly looks at the eyes, hairline, as well as the lower half of the background to determine if the image is classified as a male (when those parts are white pixels); meanwhile, it looks at the nose, cheek, and chin areas to determine if the image is classified as a female (when those parts are black pixels).
2. This makes sense to some extent, as these areas usually represents the corresponding facial features of the two sexes, for example a less emphasized chin might belong to a female face, and receding hairline might belong to a male face. But there are other aspects that does not make sense, for example, the classifier determines a picture as male if the lower half of the background has high positive values. This could be interpreted as indirect bias, possibly capturing the presence of shorter hair (often associated with males). However, this is not truly a facial feature, and it introduces the risk of misclassifying females with shorter hair as males.

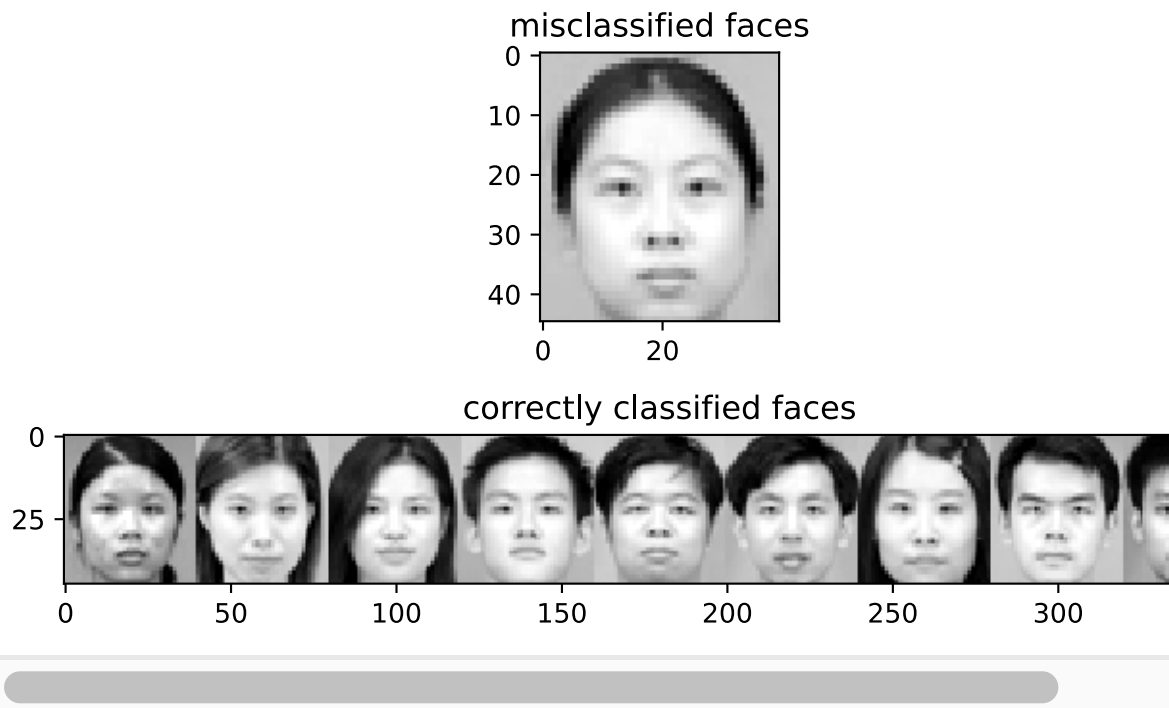
Now let's look at the misclassified faces in the test set. Run the below code to show the misclassified and correctly classified faces.

```
In [71]: # predYtest are the class predictions on the test set.

# find misclassified test images
inds = where(predYtest != testY) # get indices of misclassified test images
# make a montage
badimgs = image_montage(testX[inds], imsize=imgsize)

# find correctly classified test images
inds = where(predYtest == testY)
goodimgs = image_montage(testX[inds], imsize=imgsize)

plt.figure(figsize=(8,4))
plt.subplot(2,1,1)
plt.imshow(badimgs, cmap='gray', interpolation='nearest')
plt.title('misclassified faces')
plt.subplot(2,1,2)
plt.imshow(goodimgs, cmap='gray', interpolation='nearest')
plt.title('correctly classified faces')
plt.show()
```



Why did the classifier make incorrect predictions on the misclassified faces?

- **INSERT YOUR ANSWER HERE**

1. The classifier made incorrect predictions on the misclassified faces because the decision boundary it learned is mainly based on local pixel density rather than recognizing real features that have realistic meanings. In this case, when a female has less visible hair could lead to misclassification (e.g., the second misclassified case). Moreover, the brightness of the image as a whole greatly impacts the determination (again, because of the classifier mainly learns how to judge based on pixel density). And when an image is in general darker (e.g., the first misclassified case), it could confuse the classifier.

Support Vector Machine

Now train a support vector machine (SVM) on the same training and testing data. Use cross-validation to select the best C parameter.

```
In [72]: ### INSERT YOUR CODE HERE
paramgrid = {'C': np.logspace(-3, 3, 1000)}
svmcv = model_selection.GridSearchCV(
    svm.SVC(kernel='linear'),
    paramgrid,
    cv = 5,
    n_jobs = -1,
    verbose=True
)
svmcv.fit(trainX, trainY)
print("Best C:", svmcv.best_params_)
best_svm = svmcv.best_estimator_
```

Fitting 5 folds for each of 1000 candidates, totalling 5000 fits
 Best C: {'C': np.float64(0.004213321743847289)}

Calculate the training and test accuracy for the SVM classifier.

```
In [73]: ### INSERT YOUR CODE HERE
pred_train = best_svm.predict(trainX)
train_accuracy = metrics.accuracy_score(trainY, pred_train)

print("Accuracy on train set: {:.2f}%".format(train_accuracy*100))

pred_test = best_svm.predict(testX)
test_accuracy = metrics.accuracy_score(testY, pred_test)

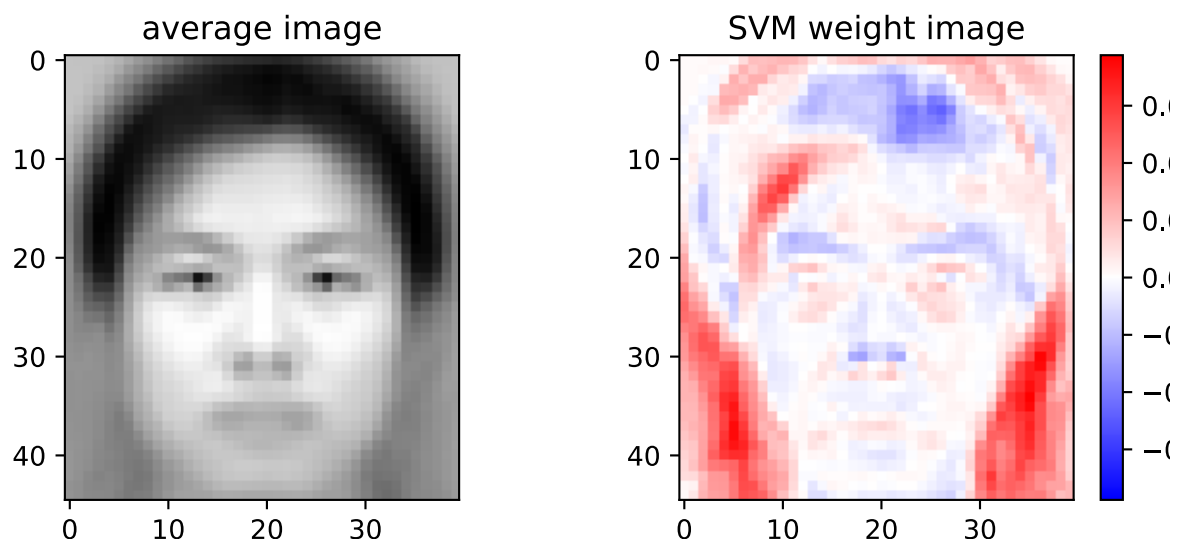
print("Accuracy on test set: {:.2f}%".format(test_accuracy*100))
```

Accuracy on train set: 90.00%

Accuracy on test set: 80.00%

Similar to before, plot an image of the hyperplane parameters w , and view the misclassified and correctly classified test images.

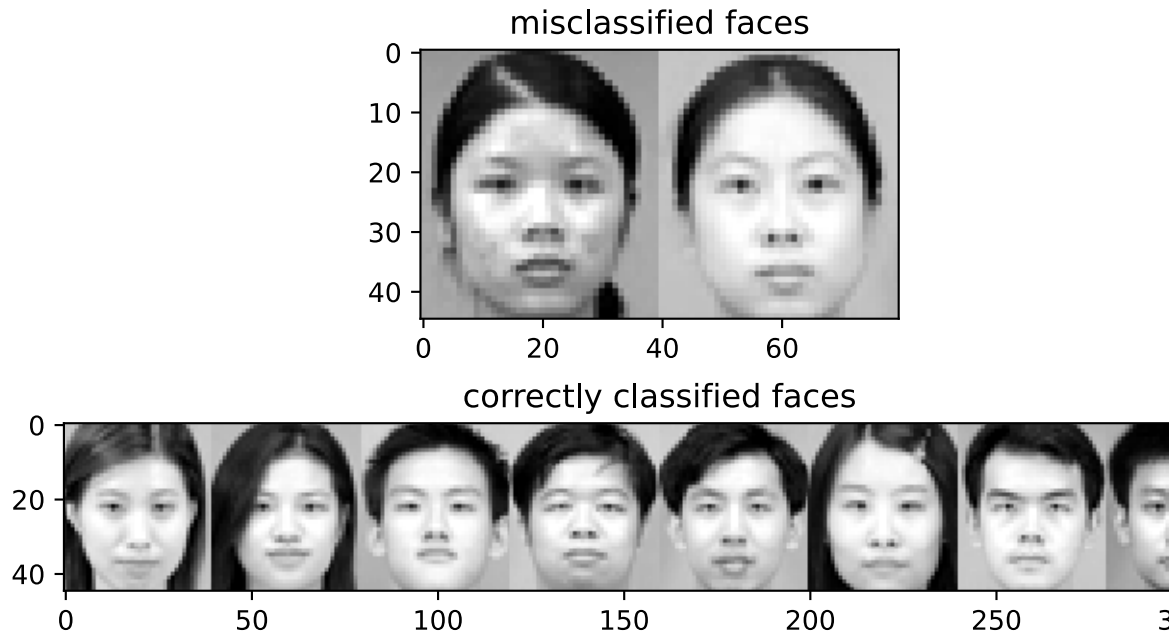
```
In [74]: ### INSERT YOUR CODE HERE
wimg_svm = best_svm.coef_.reshape(imgsize)
mycmap_svm = matplotlib.colors.LinearSegmentedColormap.from_list('mycmap',
mm_svm = np.abs(wimg_svm).max()
plt.figure(figsize=(8,3))
plt.subplot(1,2,1)
plt.title('average image')
plt.imshow(avgX.reshape(imgsize), cmap='gray')
plt.subplot(1,2,2)
plt.imshow(wimg_svm, interpolation='nearest', cmap=mycmap_svm, vmin=-mm_s
plt.colorbar()
plt.title("SVM weight image")
plt.show()
```



```
In [75]: predTest_svm = best_svm.predict(testX)
bad_idx = where(predTest_svm != testY)
good_idx = where(predTest_svm == testY)
```

```
badimgs_svm = image_montage(testX[bad_idx], imsize=imgsize)
goodimgs_svm = image_montage(testX[good_idx], imsize=imgsize)

plt.figure(figsize=(8,4))
plt.subplot(2,1,1)
plt.imshow(badimgs_svm, cmap='gray', interpolation='nearest')
plt.title('misclassified faces')
plt.subplot(2,1,2)
plt.imshow(goodimgs_svm, cmap='gray', interpolation='nearest')
plt.title('correctly classified faces')
plt.show()
```



*Are there any differences between the w for logistic regression and the w for SVM?
Is there any interpretation for the differences?*

- **INSERT YOUR ANSWER HERE**

1. From the two weight images, we can see that both LR and SVM identify similar regions of the face as important for gender classification, such as the hairline, eyes, nose and chin. However, there are several differences between the two models.
 - a. The LR weight map has a much larger range (i.e., from -0.3 to $+0.3$), while the SVM weights are much smaller (i.e., from -0.015 to $+0.015$). This is because LR directly models the probability of each class and often produces stronger coefficients, while SVM maximizes the margin between classes, leading to more regularized and balanced weights.
 - b. LR tends to produce sharper contrasts and highlights specific local features more strongly (e.g., the chin area, the nose region, etc.). In contrast, the SVM spreads weights across broader regions of the image (e.g., the entire hairline and face contour), resulting in smoother weight patterns.
2. These differences make sense given the underlying principles of the two classifiers. LR is a probabilistic model that optimizes likelihood, which can lead to large coefficients for highly discriminative features. SVM on the other hand is a

geometric model that maximizes the margin between classes, so it spreads out the influence of features more evenly.

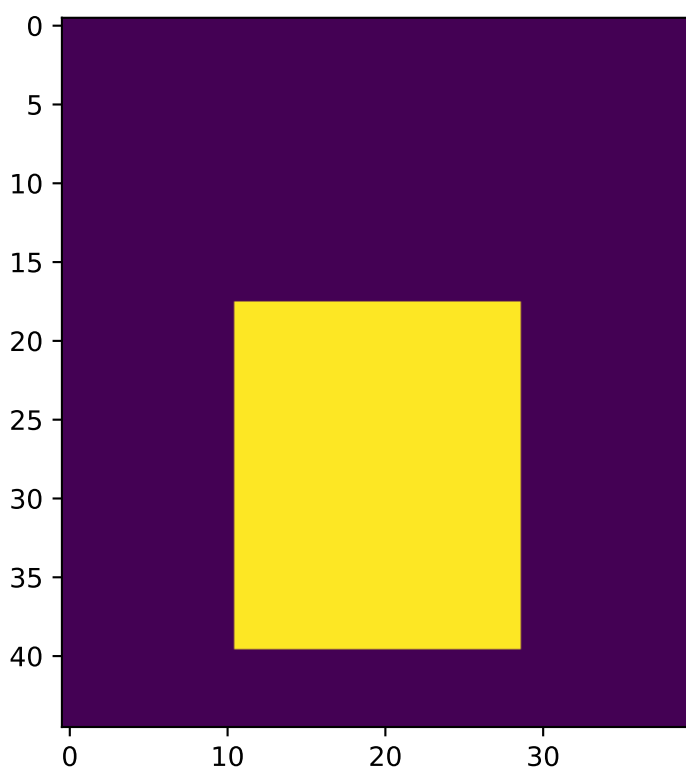
Classifying cropped faces

It seems that the hair around the face and forehead are discriminative enough to perform gender classification. Now try to perform the same task but only focusing on the face image, and not the hair.

First, we define a mask over the face.

```
In [76]: imgmask = full((img.shape), False)
#imgmask[17:41,8:32] = True
#masksize = (24,24)
imgmask[18:40,11:29] = True
masksize = (22,18)
plt.imshow(imgmask)
```

```
Out[76]: <matplotlib.image.AxesImage at 0x12dee9a90>
```



Next we crop out the face image to create the new inputs. The vectors are now 576-dim, and corresponding images are 24x24.

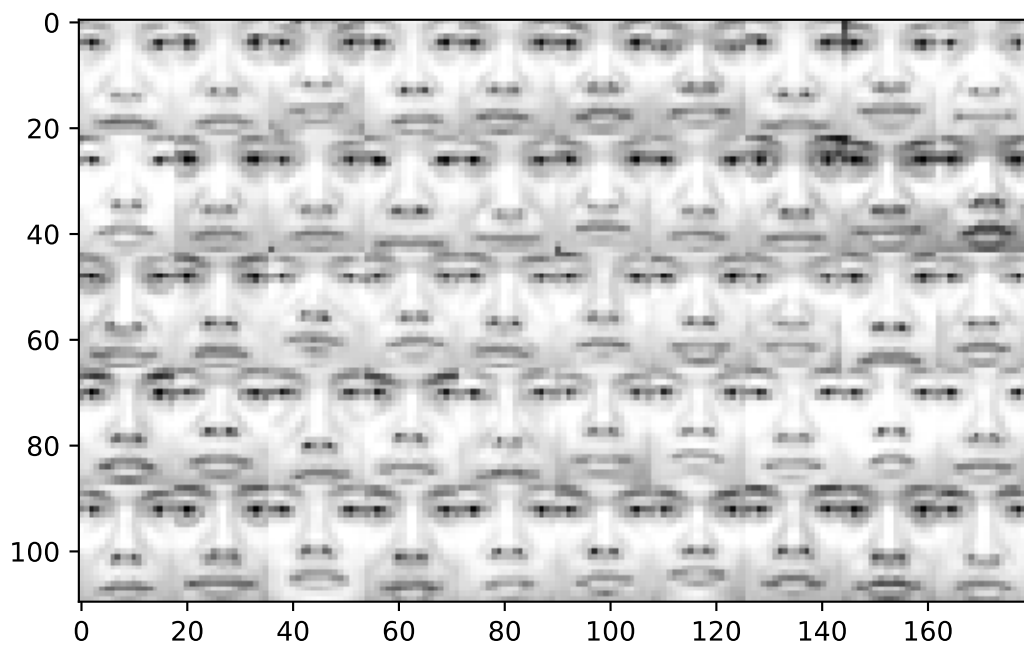
```
In [77]: Xm = X[:,imgmask.ravel()]
Xm.shape
```

```
Out[77]: (50, 396)
```

Here are the cropped images.

```
In [78]: plt.imshow(image_montage(Xm, imsize=masksize), cmap='gray', interpolation
```

```
Out[78]: <matplotlib.image.AxesImage at 0x12dea1490>
```



Generate the same training/test split as before

```
In [79]: # randomly split data into 80% train and 20% test set
trainXm, testXm, trainY, testY = \
    model_selection.train_test_split(Xm, Y,
    train_size=0.80, test_size=0.20, random_state=4487)

print(trainXm.shape)
print(testXm.shape)
```

```
(40, 396)
```

```
(10, 396)
```

```
In [80]: avgXm = avgX[imgmask.ravel()]
```

Now train logistic regression and SVM classifiers on the new cropped images.

```
In [81]: ### INSERT YOUR CODE HERE
logreg_chopped = linear_model.LogisticRegression(C=100)
logreg_chopped.fit(trainXm, trainY)

print("w =", logreg_chopped.coef_)
print("b =", logreg_chopped.intercept_)
```

```

w = [[ 1.36131970e+00 -4.77524722e-02 -1.12012466e+00 -1.34068176e+00
-1.29747138e+00 -4.43424882e-01 -5.54406106e-01  1.93026836e-03
 4.58269160e-01  2.48088591e-01  4.00961766e-01 -2.26475111e-01
 2.49674266e-01  1.86382589e-01  1.76045240e-01  5.30816891e-01
-7.31712308e-02  9.19115053e-01 -1.07457178e+00 -2.38237235e+00
-1.28986590e+00 -8.42118263e-01 -8.34492359e-01 -8.87318788e-01
-1.70721020e+00 -1.06549205e+00  5.90162637e-02  1.88366871e-01
-4.46378902e-01 -1.43733615e+00 -1.25287763e+00 -1.29756127e+00
-1.45680030e+00 -2.10720161e+00 -2.53098052e+00 -1.94267599e+00
-7.27378757e-01 -6.87003894e-01 -2.68578975e-01 -1.57299371e-02
-2.70317912e-02 -5.19300779e-01 -1.13765435e+00 -9.77101573e-01
-3.81188712e-01 -2.70007426e-01 -8.79729507e-01 -1.27325989e+00
-1.07057726e+00 -1.69018521e-01  3.52997727e-01  1.32599172e-01
 6.86879319e-03 -7.03611863e-01  1.96889328e+00  2.11775091e+00
 2.00011009e+00  1.15716014e+00  5.82181876e-01  2.13131322e-01
-2.16804624e-01  1.29803226e-02  4.62338216e-01  7.06294429e-01
 1.03941920e-01 -5.42016141e-01 -6.74165474e-02  5.94371828e-01
 6.20524482e-01  2.14091727e+00  2.77117022e+00  2.34086348e+00
-5.70590489e-01 -7.91731044e-01 -1.70929469e+00 -2.37503980e-01
-1.28475925e+00 -3.02698215e-01  2.11150947e-01  3.71100090e-02
 1.14690943e+00  1.51344853e+00  4.86767432e-01  1.17573280e-01
 3.27176741e-01 -9.03492443e-01 -2.34167756e-01 -2.05399733e+00
-1.33495197e+00  4.15664632e-01  2.40718884e-01  9.00108336e-01
 1.60573335e+00  4.48154668e-01  5.39176390e-01  7.02498253e-01
 9.19545712e-02 -6.07753090e-01  4.89686332e-02  5.45988120e-01
 1.35066949e-02  1.15734895e-01  1.90024867e+00  1.01824579e+00
 1.92178308e+00  2.51074260e+00  8.56286859e-01 -1.29814239e-01
-8.02821267e-01 -1.16125118e-01  7.53780055e-01  5.05848342e-01
 8.91071045e-01  9.04315054e-01  2.80246681e-01 -9.75361724e-01
-3.65518198e-01  2.87012081e-01 -6.48772597e-01 -3.70573458e-01
 1.27899147e+00  1.36394722e+00  6.44906688e-01  1.00594978e+00
 4.97571471e-01 -6.76652111e-01  2.92092667e-01  5.25748655e-01
 7.10014474e-01  1.24024465e+00  1.19235306e+00  1.73737000e-01
-1.50617502e-01 -1.13030273e+00 -3.73542770e-01  3.59261328e-01
-7.38512617e-01 -7.19133327e-01 -2.30313840e-01  1.14616086e+00
 1.20465963e+00  5.25779457e-01  4.31643925e-01  3.26998304e-01
 7.72385289e-01  1.60081003e+00  1.71428148e+00  1.25459182e+00
 1.15517630e-01 -4.99771347e-01 -6.49912089e-01 -1.10998048e+00
-3.63856966e-01  3.91287907e-01 -2.13406761e-01 -7.49695857e-01
-8.44924272e-01  3.80439331e-01  1.47915423e+00  1.90672537e+00
 1.73858403e+00  9.13879696e-01 -1.28254151e-01  1.52606766e-01
 1.45346045e-01 -6.67071471e-02 -3.83486215e-01 -7.46559907e-01
-5.84753462e-01 -1.02997043e+00 -2.72235535e-01  3.60262529e-01
 1.68054700e-01 -4.91064600e-01 -1.08849372e+00 -4.71384599e-01
 3.21818648e-01  4.44187052e-01  4.23432863e-01  2.68180889e-01
-2.96690733e-01  6.67941801e-02  9.89814137e-02 -2.62875935e-01
-1.84597004e-01  5.75284795e-02 -2.65685566e-01 -8.25889700e-01
 4.36333941e-02  6.73218178e-02 -4.72741574e-02  2.98131362e-01
-3.22136701e-02 -5.70725464e-01 -1.18374538e-01  3.68404182e-02
 7.46292664e-02  2.43405744e-01 -3.83397090e-01 -2.27098052e-01
-2.13310370e-01 -3.23376749e-01  6.99936848e-01  5.90400326e-01
-1.42807971e+00 -1.73139501e+00 -9.10200987e-01 -3.97306144e-01
-8.61879885e-01 -1.26528506e+00  3.34177384e-01 -7.70136310e-02
-3.08163768e-01 -1.43610254e-01 -3.16091153e-02  1.30537454e-01
-5.53265394e-01 -3.17171118e-01 -4.62618586e-01 -2.44476024e-01
 9.19659762e-01 -3.43433519e-01 -8.85146673e-01 -1.95521750e+00
-1.46483299e+00 -5.52487248e-01 -7.71217471e-01 -1.04425147e+00
-8.19727046e-02  4.79687693e-01 -6.25774578e-01 -4.88574090e-01
-6.81255912e-02  9.49009673e-02 -6.14952086e-01 -5.47701017e-01
-7.65404610e-01  5.28186882e-01  1.18979211e-01 -2.04843567e+00

```

```

-1.89709053e+00 -7.64070677e-01 -2.21696697e-01 2.39364448e-01
 3.78757113e-01 -1.68192416e+00 -5.47246189e-01 2.88787210e-02
-6.13780898e-01 -5.24845592e-01 6.31053805e-02 3.06785660e-01
-4.47469036e-01 -3.78214359e-01 -7.72476348e-02 5.27234020e-01
-7.38791227e-01 -1.00435254e+00 3.83375971e-01 5.10278425e-01
 1.06787788e+00 1.48559975e+00 1.46629924e+00 -1.04746201e+00
-9.49306486e-01 -1.08815212e+00 -5.07381727e-01 5.96107415e-03
 7.25683241e-02 -2.47677967e-01 -2.98550852e-01 -2.34259827e-02
 1.32418395e-01 -2.53076018e-01 -9.28149609e-01 -1.06840533e+00
-4.61880277e-01 -1.02383384e-01 3.19129346e-01 6.94122189e-01
 1.76265443e-01 1.82263809e-01 -9.26229130e-01 -1.07393344e+00
-4.67348397e-01 1.72384432e-01 -4.12183732e-02 -6.74922899e-01
-1.33219315e-01 2.27804492e-01 -9.33615855e-03 -5.09928631e-01
-5.66524893e-01 -5.90885189e-01 1.13911952e-01 3.01910899e-01
 9.63143706e-02 1.92919964e-01 5.04180436e-02 -1.54010090e-01
-2.36707987e-01 -3.83745729e-02 -1.62713339e-01 -6.92779190e-02
 3.14115554e-02 -3.78725649e-01 1.37726561e-01 5.90151313e-01
-2.32574959e-01 5.51800741e-01 1.61505427e+00 1.43972110e+00
 1.57051264e+00 1.32122547e+00 4.48069481e-01 1.03500705e-01
 4.90588998e-01 3.15900500e-01 5.22098467e-01 7.31778407e-01
 9.41969236e-01 5.16799743e-01 2.92164484e-01 1.98286281e-02
 1.05944980e-01 4.93284669e-01 -3.99680906e-02 9.47476923e-01
 2.00230352e+00 3.49177777e-01 1.24681961e+00 6.22001841e-01
-2.52041503e-01 1.34164909e-01 8.53508015e-01 7.65533707e-01
 9.41304456e-01 1.32594687e+00 4.69121870e-01 6.47913298e-01
 2.74507374e-01 -1.24063980e-01 -7.47323562e-02 3.65395675e-01
 2.81987246e-01 2.36313457e-01 2.97467120e-01 4.21356305e-01
 9.94299466e-02 1.20623960e-01 1.92104526e-02 6.16155553e-01
 2.80830413e-01 -7.24501620e-01 -8.40250967e-01 -2.81012691e-01
 6.23169409e-02 4.33063200e-01 -3.93473216e-02 -4.60415221e-01
 5.35976611e-02 4.99373276e-01 1.42739229e+00 6.65062161e-01
-6.35111327e-01 -5.48659718e-01 5.35651759e-02 4.55599304e-01
-7.43615995e-02 -1.29417355e-01 2.57053319e-02 -4.51425668e-01
-1.62784700e-01 7.19239996e-01 3.37362776e-01 5.25468170e-01
-7.17838455e-02 -7.43775963e-01 4.92467852e-01 4.05623061e-01
 1.09229643e+00 3.17207616e-01 -2.61254201e-01 -1.08379318e+00
-1.11368496e+00 -8.25150847e-01 -8.12365616e-01 -1.14161796e+00
-1.18124282e+00 -9.22481463e-01 -3.31171338e-01 6.21572236e-01
 5.40408405e-01 6.95973259e-01 -5.73228994e-01 -9.20060198e-01]]
b = [1.58024162]

```

```

In [82]: ### INSERT YOUR CODE HERE
predY_chopped = logreg_chopped.predict(testXm)
accuracy_chopped = metrics.accuracy_score(testY, predY_chopped)
print("Accuracy on test set: {:.2f}%".format(accuracy_chopped*100))

```

Accuracy on test set: 80.00%

Compute the train/test accuracies of the two classifiers.

```

In [83]: ### INSERT YOUR CODE HERE
train_predY_chopped = logreg_chopped.predict(trainXm)
train_accuracy_chopped = metrics.accuracy_score(trainY, train_predY_chopped)
print("Accuracy on train set: {:.2f}%".format(train_accuracy_chopped*100))

test_predY_chopped = logreg_chopped.predict(testXm)
test_accuracy_chopped = metrics.accuracy_score(testY, test_predY_chopped)
print("Accuracy on test set: {:.2f}%".format(test_accuracy_chopped*100))

```

Accuracy on train set: 100.00%
Accuracy on test set: 80.00%

```
In [84]: param_grid_chopped = {'C': np.logspace(-3, 3, 1000)}
svm_cv_chopped = model_selection.GridSearchCV(
    svm.SVC(kernel='linear'),
    param_grid_chopped,
    cv = 5,
    n_jobs = -1,
    verbose=True
)
svm_cv_chopped.fit(trainXm, trainY)
print("Best C:", svm_cv_chopped.best_params_)
best_svm_chopped = svm_cv_chopped.best_estimator_
```

Fitting 5 folds for each of 1000 candidates, totalling 5000 fits
Best C: {'C': np.float64(0.36187498124112805)}

```
In [85]: train_predY_svm_chopped = best_svm_chopped.predict(trainXm)
train_accuracy_svm_chopped = metrics.accuracy_score(trainY, train_predY_s
print("Accuracy on train set: {:.2f}%".format(train_accuracy_svm_chopped*

test_predY_svm_chopped = best_svm_chopped.predict(testXm)
test_accuracy_svm_chopped = metrics.accuracy_score(testY, test_predY_svm_
print("Accuracy on test set: {:.2f}%".format(test_accuracy_svm_chopped*10
```

Accuracy on train set: 95.00%
Accuracy on test set: 70.00%

Compare the classification performance? Which performs better and why?

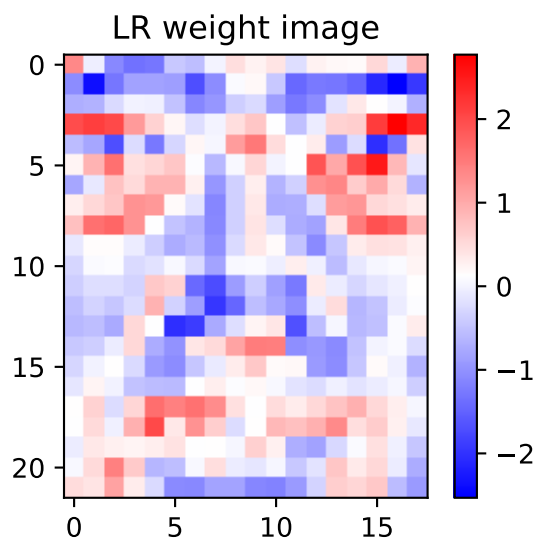
- **INSERT YOUR ANSWER HERE**

1. For LR, the training accuracy reached 100% while the testing accuracy reached 80% for both full and cropped images. This indicates that logistic regression tends to a. overfit the training set; and b. cropping the images did not reduce the overall performance.
2. For SVM, the training accuracy was 90% with a testing accuracy of 80% on the full images, which means a relatively balanced generalization. When trained on cropped images, the training accuracy increased to 95% while the testing accuracy dropped to 70%, which suggests stronger overfitting and reduced generalization ability.
3. This shows that classifiers trained on full images performed better, particularly in the case of SVM models. Although cropping reduces irrelevant background information, it also removes potential useful contextual information such as hairline and hairstyle. These information may not be classified as facial features, but they still contribute discriminative information for gender classification in this dataset. In other words, judging based purely on facial features poses more significant challenges for models.

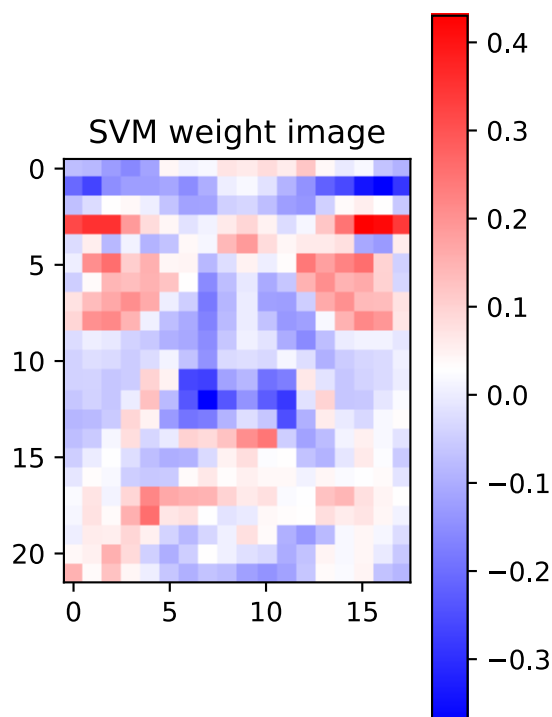
Visualize the weights as an image and interpret what discriminative information each classifier is using.

```
In [86]: ### INSERT YOUR CODE HERE
w_logreg = logreg_chopped.coef_.reshape((22, 18))
mycmap = matplotlib.colors.LinearSegmentedColormap.from_list('mycmap', [
plt.figure(figsize=(8,3))
plt.subplot(1,2,1)
plt.imshow(w_logreg, cmap=mycmap)
plt.colorbar()
plt.title("LR weight image")
```

Out[86]: Text(0.5, 1.0, 'LR weight image')



```
In [87]: ### INSERT YOUR CODE HERE
w_svm = best_svm_chopped.coef_.reshape((22, 18))
plt.subplot(1,2,2)
plt.imshow(w_svm, cmap=mycmap)
plt.colorbar()
plt.title("SVM weight image")
plt.show()
```



- **INSERT YOUR ANSWER HERE**
- From the comparison of the two weight maps, we can see that LR has a stronger emphasize on the mouth area, with distinct positive weights contouring the mouth shape. In comparison, the SVM shows emphasized negative weights on the nose area.

Custom kernel SVM

Now we will try to use a custom kernel with the SVM (for classifying cropped faces). We will consider the following RBF-like kernel based on L1 distance (i.e., cityblock or Manhattan distance),

$$k(\mathbf{x}, \mathbf{y}) = \exp\left(-\alpha \sum_{i=1}^d |x_i - y_i|\right)$$

where x_i, y_i are the elements of the vectors $\mathbf{x}, \mathbf{y}$, and α is the hyperparameter. The difference with the RBF kernel is that the new kernel uses the absolute difference rather than the squared difference. Thus, the new kernel does not "drop off" as fast as the RBF kernel using squared distance.

- Implement the new kernel as a custom kernel function, `svm.SVC` can accept a self-defined callable kernel function as the kernel argument. The `scipy.spatial.distance.cdist` function will be helpful.
- Train the SVM with the new kernel. To select the hyperparameter α , you need to run cross-validation "manually" by: 1) trying different values of α , and running cross-validation to select C ; 2) selecting the α with the highest cross-validation score `best_score_` in `GridSearchCV`.

```
In [88]: ### INSERT YOUR CODE HERE
from scipy.spatial.distance import cdist

def laplacian_kernel(alpha):
    def _kernel(X, Y):
        D = cdist(X, Y, metric='cityblock')
        return np.exp(-alpha * D)
    return _kernel
```

```
In [89]: ### INSERT YOUR CODE HERE
alphas = np.logspace(-3, 1, 100)
Cs      = np.logspace(-3, 3, 100)
best = {
    "score": -np.inf,
    "alpha": None,
    "C": None,
    "clf": None
}

for a in alphas:
    ker = laplacian_kernel(a)
    grid = model_selection.GridSearchCV(
```

```

svm.SVC(kernel=ker),
param_grid={"C": Cs},
cv=5, n_jobs=-1, verbose=False
)
grid.fit(trainXm, trainY)
if grid.best_score_ > best["score"]:
    best["score"] = grid.best_score_
    best["alpha"] = a
    best["C"] = grid.best_params_["C"]
    best["clf"] = grid.best_estimator_

print("Best alpha:", best["alpha"])
print("Best C:", best["C"])
print("Best CV score: {:.2f}%".format(best["score"] * 100))

```

Best alpha: 0.001
 Best C: 40.37017258596558
 Best CV score: 90.00%

```

In [90]: pred_tr = best["clf"].predict(trainXm)
         pred_te = best["clf"].predict(testXm)

         acc_tr = metrics.accuracy_score(trainY, pred_tr)
         acc_te = metrics.accuracy_score(testY, pred_te)

         print("Accuracy on train set: {:.2f}%".format(acc_tr * 100))
         print("Accuracy on test set: {:.2f}%".format(acc_te * 100))

```

Accuracy on train set: 97.50%
 Accuracy on test set: 90.00%

Does using the new kernel improve the results?

- **INSERT YOUR ANSWER HERE**

- Yes. The custom kernel based on the L1 distance improves the results compared to the standard SVM. With the standard SVM on cropped images, the test accuracy was 70.00%, while with the custom kernel, the test accuracy improved to 90.00%.
- This means that L1-based kernel can capture more nuanced features of facial structures, and is more robust to general variations such as lighting, which means it has a better generalization.